Senior Full Stack .NET Developer (Tech Lead) - Take-Home Exercise

Time limit: 2 hours maximum

AI assistance: Candidates are expected and encouraged to use AI tools (Copilot, ChatGPT, Cursor, etc.)

Submission: A Git repository (GitHub/Azure DevOps) or a zip file containing all artefacts

---

What We're Assessing

AI tools can scaffold a .NET project, generate CRUD endpoints, and write boilerplate in minutes. This exercise is designed to test what AI alone cannot do well:

- **Judgement** - making the right technology and design trade-offs for a real customer context
- **Full-stack thinking** - API, front-end, database, and infrastructure as a coherent whole
- **Code quality** - clean, testable, production-ready code (not just code that compiles)
- **Customer lens** - solving business problems, not just technical ones
- **AI fluency** - using AI tools effectively and integrating AI capabilities into solutions

We value clear thinking over volume. A well-reasoned incomplete submission beats a comprehensive but shallow one.

Scenario

Brownfield Consulting has engaged Datacom to build an internal

Knowledge Base application for their 200-person consulting firm. Consultants need a central place to capture, search, and share reusable knowledge - project lessons learned, technical guides, client-safe templates, and best practices.

They currently use a shared drive with 2,000+ unstructured Word docs and PDFs that nobody can find anything in.

Core Requirements

Requirement	Detail
Create articles	Consultants can author knowledge articles with a title, rich-text

Requirement	Detail
	body, tags, and optional file attachments
Search	Full-text search across articles, with AI-powered "smart search" that understands intent (e.g. searching "how to handle difficult stakeholders" finds an article titled "Client Relationship Management Strategies")
Categorise	Articles belong to categories (e.g. "Project Management", "Technical", "Sales"). AI should suggest categories when an article is created
Quality signals	Articles can be upvoted, and the system tracks view counts. Search results should factor in quality signals
Access control	All consultants can read. Authors can edit their own articles. Admins can edit/delete any article
Audit trail	Track who created, edited, and viewed each article

Non-Functional Requirements

- 200 concurrent users maximum
- Sub-second search response for the first page of results
- Hosted on Azure
- Must integrate with Azure AD (Entra ID) for authentication
- The customer wants to see a working demo in 4 weeks - this is the MVP

Technology Constraints (from customer)

- Back end must be .NET (C# / ASP.NET Core) - their internal team will maintain it
- They have a SQL Server license they want to use
- Front end: customer is open to React or Blazor - recommend what you'd use and why
- AI features should use Azure OpenAI Service (they already have an Azure OpenAI resource provisioned)

Deliverables

1. Solution Architecture (required - ~30 min)

Produce a brief architecture document that covers:

- **Architecture diagram** - show the major components (front end, API, database, AI service, auth, storage) and how they connect. Any format - draw.io, Mermaid, ASCII, or a sketch photo.
- **Technology choices** - for each component, state what you'd use and why (e.g. why Blazor over React or vice versa, which Azure services, search approach)

- **AI integration design** - how does the “smart search” work? How does auto-categorisation work? Be specific about the Azure OpenAI integration (embeddings, completions, or both? Where are vectors stored? Sync or async processing?)
- **Data model sketch** - key entities and their relationships (not full DDL - a diagram or a bullet list is fine)
- **MVP scope** - given the 4-week deadline, what do you build first and what do you defer?

Format: Markdown, PDF, or annotated diagrams - whatever communicates most clearly.

2. Working API (required - ~50 min)

Build a small but functional ASP.NET Core Web API that demonstrates your code quality and design approach. Implement:

- **POST /api/articles** - create a new article (title, body, tags)
- **GET /api/articles/{id}** - retrieve an article by ID (include view count increment)
- **GET /api/articles?search=term&category=xyz&page=1&pageSize=20** - search/filter with pagination
- **POST /api/articles/{id}/upvote** - upvote an article

Requirements:

- Use Entity Framework Core with a SQL Server (or SQLite for portability) data context
- Include at least one example of proper input validation
- Include structured error responses
- Use dependency injection appropriately
- The code should be buildable (but doesn’t need to run end-to-end without a database - we’ll read the code)

We’re not looking for a complete application. We’re looking at:

- How you structure a solution
- Your C# code style and patterns
- How you handle edge cases and errors
- Whether the code is testable

3. Test Examples (required - ~15 min)

Write **2-3 unit or integration tests** that demonstrate your testing approach. Choose the tests that show the most about how you think about quality:

- At least one unit test for a piece of business logic

- At least one integration test for an API endpoint (using `WebApplicationFactory` or equivalent) is a bonus but not required

We're looking at what you choose to test and how, not volume.

4. Technical Recommendation (required - ~15 min)

Write a short (half-page to one-page) **customer-facing recommendation** for the AI-powered search approach. Imagine you're presenting this to the Greenfield Consulting CTO, who is technical but not an AI specialist.

Cover:

- What approach you recommend and why (in plain language)
- What it will and won't do well (manage expectations)
- Rough cost implications (Azure OpenAI is pay-per-token - what does that mean for 200 users and 2,000+ articles?)
- Any risks or caveats

This tests your ability to communicate technical decisions to a customer - a core part of the Tech Lead role.

5. Bonus (optional - if time allows)

Pick **one** of the following:

- **Front-end component sketch** - mock up or describe the search results page. What does the user see? How do search results display relevance? How do quality signals (upvotes, views) appear? This can be a wireframe, a React/Blazor component skeleton, or a written description.
- **CI/CD pipeline design** - describe or write a YAML pipeline (GitHub Actions or Azure DevOps) that builds, tests, and deploys the API to Azure. Focus on the stages and quality gates, not syntax perfection.
- **Database migration strategy** - how would you migrate the 2,000 existing Word docs and PDFs into the knowledge base? What's the process, and how does AI help?

Submission Structure

```
submission/
├── README.md                    # How to read your submission, assumptions, what you'd do with more time
├── architecture/
│   └── design.md                # Architecture description, diagrams, data model, MVP scope
├── src/
│   └── KnowledgeBase/           # ASP.NET Core solution
│       ├── KnowledgeBase.Api/   # Web API project
│       └── KnowledgeBase.Tests/ # Test project
├── recommendation/
│   └── ai-search-recommendation.md  # Customer-facing technical recommendation
└── bonus/                       # (optional)
    └── ...
```

Evaluation Criteria

Area	Weight	What we're looking for
Working API & code quality	30%	Clean C#, appropriate patterns, error handling, testability, EF Core usage
Solution architecture	25%	Coherent full-stack design, justified choices, AI integration design, realistic MVP scoping
Testing approach	15%	Meaningful test selection, good assertions, demonstrates quality mindset
Customer recommendation	15%	Clear communication, honest trade-offs, appropriate level of detail for the audience
Overall communication	15%	README quality, assumptions documented, well-organised submission

What will impress us

- Code that reads like it was written for a team to maintain, not just to pass the exercise
- Asking clarifying questions in your README (shows you know what you don't know)
- Honest MVP scoping - what you'd cut to hit 4 weeks, and why
- A recommendation that a non-AI-specialist CTO could actually act on
- Thoughtful AI integration design that goes beyond "call the OpenAI API"

What won't impress us

- A sprawling solution with 15 projects and no substance
- Obviously AI-generated code that hasn't been reviewed (e.g. inconsistent patterns, unused usings, boilerplate comments everywhere)
- Ignoring the customer context - over-engineering for 200 users or gold-plating the MVP
- "Happy path only" code with no validation or error handling
- A recommendation full of jargon that the customer couldn't understand

AI Usage Policy

You **should** use AI tools. We expect it. Here's what matters:

- **You must understand and be able to defend everything in your submission.** In a follow-up conversation, we will ask you to explain your code, modify the architecture under new constraints, and walk through your design decisions.
- **AI is a tool, not a substitute for judgement.** We can tell when someone has prompted "build me a .NET knowledge base API" and submitted the output. The scenario has enough nuance that generic answers will fall flat.
- **Credit your tools.** Briefly note in your README which AI tools you used and how (e.g. "Used Copilot for EF Core scaffolding, wrote business logic and tests manually").

Logistics

- **Time:** 2 hours maximum. We mean it - we'd rather see prioritised, thoughtful work than a rushed attempt at completeness. If you run out of time, tell us what you'd do next in your README.
- **Questions:** If anything is unclear, note your assumption and proceed. Identifying ambiguity is a positive signal.
- **Follow-up:** Candidates will be invited to a 1 hour conversation to discuss their submission as part of a technical interview. This is collaborative, not adversarial - we want to see how you think, not catch you out.